EC-241 Object-oriented Programming

LECTURE 7

1

OPERATOR OVERLOADING

- Giving the normal C++ operators such as +, -, * , <=, and += etc. , additional meanings when they are applied to user-defined types (i.e. classes)
- You cannot create new operators like *& and try to overload them; only existing operators can be overloaded

2

- Operator overloading allows us to call

```
a+b*c
```

- on class objects rather than calling

```
add( a, multiply (b,c));
```

- Assuming the precedence of operators

3

OPERATOR OVELOADING

- The precedence of an operator cannot be changed by overloading
- The associativity (i.e. left-to-right or right-to-left) of an operator cannot be changed
- The arity (i.e. no. of operands) of an operator cannot be changed
- The meaning of how an operator works on built-in types cannot be changed
  - Operator overloading works only on objects of user defined types or with a mixture of user-defined and built-in types
  - At least one argument of an operator function must be a class object or a reference to a class object

4

Addition operator overloading

```
Time operator+(const Time& lhs, const Time&
rhs)
{
     Time temp = lhs;
     temp.seconds += rhs.seconds;
     temp.minutes += temp.seconds / 60;
     temp.seconds %= 60;
     temp.minutes += rhs.minutes;
     temp.hours += temp.minutes / 60;
     temp.minutes %= 60;
     temp.hours += rhs.hours;
     return temp;
}
```

5

OVERLOADING UNARY OPERATORS

```
class Counter{
private:
        int count;
public:
        Counter():count(0)
        {       }
        int getcount()
        {return count;}
void operator ++()
{       ++count;        }
};      // END OF CLASS
```

```
void main()
{
        Counter c1, c2;
        cout<<\nc1="<<c1.getcount();
        cout<<"\nc2="<<c2.getcount();
        ++c1; //prefix
        ++c2; //prefix
        ++c2; //translated to
              // c2.operator++() by compiler
cout<<\nc1="<<c1.getcount();
cout<<"\nc2="<<c2.getcount();
```

6

OPERATOR RETURN VALUES

In the previous example, you cannot write

```
c1=++c2;      //Error; Why?

Counter operator ++()
{
   ++count;
   Counter temp;
   temp.count=count;
    return temp;
}
```

7

OVERLOADING BINARY OPERATORS

```
class complex {
private:
        int real, imag;
public:
complex(): real(0),  imag(0) {}
complex(int re, int im): real(re),
                imag(im)    {   }
 void setdata()
{    cin>>real; cin>>imag;    }
 void showdata() const
{ cout<<real<<'+'<<imag<<'i'; }
complex operator + (complex);
};
```

```
complex complex::operator + (complex c)
{ //note: we aim not to change the operands
        int re2=real + c.real;
       int im2=imag+c.imag;
        return complex(re2, im2)
}
 void main()
{      complex c1, c2(1,2), c3, c4;
       c1.setdata();
        c3=c1+c2; //same as c1.operator + (c2)
        c4 = c1+c2+ c3; //note cascading
       c4. showdata();
}
```

8

In

c3=c1+c2;

- The argument on the LEFT side of the operator (c1 in this case) is the object of which the operator is a member.
- The argument on the RIGHT side of the operator (c2 in this case) must be furnished as an argument to the operator
- The operator returns a value which can be assigned or used in other ways; in this case it is assigned to c3
- An overloaded operator implemented as a member function always requires one less argument than its no. of operands, since one operand is the object of which the operator is a member. That is why unary operators require no arguments.

9

CLASS WORK

Given the following class, incorporate the comparison operator (>)

```
class myclass {
        int a;
        …
};
```

10

Less than (<) operator

```
class pair
{
     public:
     int x,y;
     bool operator < ( const pair& p )
const
     {
          if(x==p.x) return y<p.y;
          return x<p.x;
     }
};
```

11

ASSIGNMENT OPERATOR

- The (default) assignment operator (=) may be used with every class without explicit overloading.
- The default behavior of the assignment operator is a *member-wise assignment* of the data members of the class, which is sufficient for a statically allocated data structure.

12

Assignment by Default Member-wise Copy

```
class Date {
public:
  Date( int = 1, int = 1, int = 1990 ); // default constructor
  void print();
private:
  int month;
  int day;
  int year;
};
// Simple Date constructor with no range checking
Date::Date( int m, int d, int y )
{
  month = m;
  day = d;
  year = y;
}
```

13

```
// Print the Date in the form mm-dd-yyyy
void Date::print()
  { cout << month << '-' << day << '-' << year; }

int main()
{
  Date date1( 7, 4, 1993 ), date2;  // d2 defaults to 1/1/90
  cout << "date1 = ";
  date1.print();
  cout << "\ndate2 = ";
  date2.print();
  date2 = date1;   // assignment by default memberwise copy
  cout << "\n\nAfter default memberwise copy, date2 = ";
  date2.print();
  cout << endl;

  return 0;
}
```

```
date1 = 7-4-1993
date2 = 1-1-1990

After default memberwise copy, date2 = 7-4-1993
```

14

Criticism

- Operator overloading has often been criticized
- It allows programmers to give operators completely different semantics
- `a << 1;`
- Shifts the bits in a by 1 bit IF a is integer
- If a is output stream, print a on screen.
- This can catch subsequent programmers.
- Java developers decided not to use this feature

15